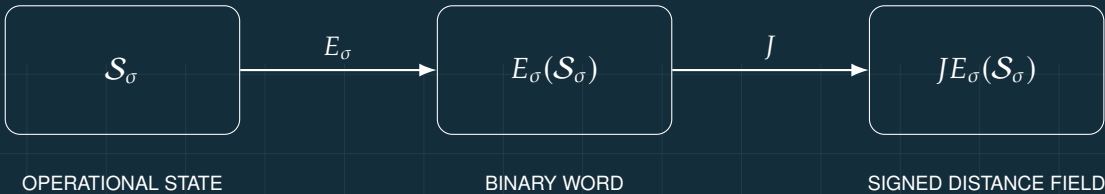


Unified Field Theory

Tom Klootwijk
A.D. Peereboom

Referential substrate · Seed-field dynamics
Binary closure · Exact state representation



Unified formal corpus

The *Tom Klootwijk – A.D. Peereboom Unified Field Theory* organizes the two supplied works into a single mathematical architecture. UGTS-KC 3.6 supplies the literal definition substrate: typed objects, dependency graphs, geometry, topology, hinges, support, compatibility, events and lineage. Gambit Formal Closure 4.0 supplies a concrete seed–field transition: immutable lookup geometry, conservative graph transport, local reaction, recurrent activity, geometric memory, Boolean output and a sufficient restart state.

The common object is a field-bearing state whose definitions are explicit and whose evolution has an ordered transition law. A fixed definition registry names that law; an operational encoding represents its state as a binary word; an injective geometric encoding represents that word as an exact signed distance field. The central relations are

$$\begin{aligned} T_\sigma : \mathcal{S}_\sigma &\longrightarrow \mathcal{S}_\sigma, & F_\sigma E_\sigma &= E_\sigma T_\sigma, & \Psi_\sigma J &= J F_\sigma. \\ \mathcal{T}_\sigma(\mathcal{D}_\sigma, S) &= (\mathcal{D}_\sigma, T_\sigma S), & B &= 118N + 27. \end{aligned}$$

The first relation states invariant-preserving evolution. The second states agreement between operational and binary representations. The third states agreement between binary and signed-distance representations. The final line joins the dynamic state to its fixed, content-addressed definition registry.

36	20	16	83
Operator entries	Pattern recipes	Supporting proofs	Executed tests

Edition conventions

References of the form [S1, section] and [S2, section] identify the supplied PDFs. Their titles, page correspondence and SHA-256 digests appear in Appendix D. Mathematical statements inherited from those PDFs retain their source labels, including U4 and C1–C16. The F-series identifies constructions made in this edition: the unified product construction, the canonical record recipe, the example registry, the generated Dutch profile, the Python transcription, the explicit fixture seed, the replay envelope and the executed tests.

The archive contains this PDF, its editable L^AT_EX manuscript, JSON and CSV specifications, a 100-row language profile, executable Python modules, test records, a replay example and package checksums. The two input works are registered by filename and digest; the archive’s report and executable artifacts belong to this edition.

Notation

The registry is written $\mathcal{D}$; the numerical denominator is $D = 65536$. The letter A denotes the number of angular columns in the numerical specialization, while a typed map’s domain is written $\mathcal{A}$. An iteration index n labels the trajectory; mutable phase, level and cooldown are stored explicitly in the state. Ordered compositions act right to left. All bit fields are least-significant-bit first unless a displayed numeral is explicitly written in ordinary reading order.

Contents

Unified formal corpus	i
1 Unified ontology and formal commitments	1
1.1 Corpus correspondence	1
1.2 World object	1
1.3 Definition records and instances	1
1.4 Referential queries	2
2 Geometry, topology, hinges and events	3
2.1 Parametric and implicit field definitions	3
2.2 Radix shells and local log-polar charts	3
2.3 Pulse geometry and parity occupancy	3
2.4 Explicit gluing and sheet state	4
2.5 Generic hinge and composition	4
2.6 Kinematics, guards and verified events	5
2.7 Normative substrate schedule	5
3 Dutch number-order and pulse calculus	6
3.1 Profile and typed representations	6
3.2 Place order and spoken order	6
3.3 Pulse projection and active-bit comparison	6
3.4 Worked instance: nineteen	7
3.5 Worked instance: twenty-three	7
3.6 The generated atlas	7
4 Complete seed and frozen field geometry	8
4.1 Generative specification	8
4.2 Paired grammar and finite generation	8
4.3 Log-polar sample chart	8
4.4 Quantized field and packed texel	8
4.5 The edition's explicit fixture seed	9
5 Operational state and normative transition	10
5.1 Invariant domain	10
5.2 Stage 1: read the previous output and sample	10
5.3 Stage 2: gated conservative graph transport	11
5.4 Stage 3: conservative local reaction	11
5.5 Stage 4: sensing, recurrence and geometric memory	11
5.6 Stage 5: one-bit codec and persistent hinge	12
5.7 Stage 6: packing and bounded global control	12
6 Unified Closure Theorem U4 and supporting proofs	13
6.1 The theorem	13
6.2 C1. Initialization and finite addressing	13
6.3 C2. Finite generative closure	13
6.4 C3. Shared edge law and nonnegative transport	14
6.5 C4. Local reaction preserves total material	14
6.6 C5. Recurrence, memory and arithmetic bounds	14
6.7 C6. One-bit codec closure and prefix identity	14
6.8 C7. Hysteresis and event closure	15
6.9 C8. Exact packing and parity algebra	15
6.10 C9. Phase and cooldown form a sufficient quotient	15
6.11 C10. Boolean arithmetic refinement	16
6.12 C11. Canonical state word and checkpoint sufficiency	16
6.13 C12. Autonomous closure and eventual recurrence	16

6.14	C13. Exact signed-distance representation	17
6.15	C14. Exact reduced-description criterion	17
6.16	C15. Causal self-reference	17
6.17	C16. Staged parallel refinement	18
7	The unified referential seed–field construction	19
7.1	Registering the operational law	19
7.2	Unified Field Representation Theorem UFT-1	19
7.3	Working field and encoded-state field	20
7.4	Lineage and the operational projection	20
7.5	Field, memory and self-reference	21
8	Canonical representation and replay files	22
8.1	Operational field order	22
8.2	The F-CAPSULE-1 envelope	22
8.3	Load and validation sequence	23
8.4	Composing segment certificates	23
8.5	Exact field encoding example	23
9	Executable reference, examples and verification	24
9.1	Artifact organization	24
9.2	Executable parameter contract	24
9.3	The executed fixture trajectory	25
9.4	One-read feedback example	25
9.5	Current test inventory	25
9.6	Source release records	26
9.7	Reproduction commands	26
9.8	Formal synthesis	26
A	Complete UGTS-KC operator catalogue	27
B	Complete pattern recipe library	29
C	Complete F-NL-1 number profile	30
D	Source register and edition crosswalk	32
D.1	Input works	32
D.2	Edition constructions	32
D.3	Content correspondence	33
D.4	Archive map	33

1 Unified ontology and formal commitments

1.1 Corpus correspondence

UGTS-KC 3.6 is organized around the query-first chain

grammar $\rightarrow$ typed state $\rightarrow$ support $\rightarrow$ compatibility $\rightarrow$ guard $\rightarrow$ event $\rightarrow$ transition $\rightarrow$ lineage.

Its defining addition is a registry of literal definitions inside the substrate. Gambit 4.0 is organized around a complete seed and a closed transition over a product state. The unification identifies the Gambit step with one registered transition definition, while retaining the geometry, language, topology and event operators as typed components of the same formal vocabulary. [S1, §§1, 3, 7] [S2, §§1–4]

Component	Referential substrate	Numerical specialization
Definition	Typed, content-addressed record	Transition, seed, field decoder and state encoder
Instance	Literal parameters and typed state	Cells and bounded global control
Field	Parametric or implicit definition	Frozen signed field code and unsigned drive
Support	Local admission predicate	Both sampled edge endpoints have non-positive fields
Compatibility	Sheet, phase, branch or policy test	Shared edge law selected by incoming latches
Hinge	Continuous, discrete or connector map	Hysteretic persistent latch
Transition	Guarded, ordered state update	Conservative integer update T_σ
Lineage	Event and parent/child records	Ordered run observations and restart endpoints

Table 1: A typed correspondence. The two scheduling layers retain their own declared order.

1.2 World object

Definition 1.1 (Referential world). The substrate world is

$$\mathcal{W} = (\mathcal{D}, \mathcal{X}, \mathcal{G}, \mathcal{R}, \mathcal{S}, \mathcal{C}, \mathcal{H}, \mathcal{E}, \mathcal{T}, \mathcal{I}, \mathcal{L}, \mathcal{P}).$$

Here $\mathcal{D}$ is the definition registry, $\mathcal{X}$ the instance family, $\mathcal{G}$ the bounded grammar, $\mathcal{R}$ the relations and fields, $\mathcal{S}$ the support predicates, $\mathcal{C}$ compatibility, $\mathcal{H}$ hinges, $\mathcal{E}$ events, $\mathcal{T}$ transitions, $\mathcal{I}$ invariants, $\mathcal{L}$ lineage and novelty records, and $\mathcal{P}$ phase-ordered pipelines.

A world therefore contains both the objects being evaluated and the definitions by which they are evaluated. A query may return a state value, the definition responsible for it, the dependencies of that definition or the sequence of registered operations that produced it. [S1, §3.1]

1.3 Definition records and instances

Definition 1.2 (Literal definition). A definition is the tuple

$$d = (\text{id}, \text{kind}, \mathcal{A}, \mathcal{B}, \theta, \text{Dep}, p, \text{cap}, \text{inv}, \text{prov}, h).$$

The fields specify an ID, a kind, a typed domain and codomain, literal parameters, dependency IDs, an evaluation phase, capabilities, invariants, provenance and a content address. Its content address is

$$h = \text{SHA256}(\text{canonicalJSON}(d \setminus \{h\})).$$

An instance is $x = (\text{id}, \text{definition_ref}, \text{literal}, \text{state})$.

An operator family and its concrete invocation are separate objects. For example, a regular-polygon definition can accept a pulse count and radius, while the instance for 19 carries a literal value and a three-pulse sequence. The instance’s identity survives a change of coordinates or a different geometric rendering. [S1, §3.2]

The canonical recipe used by the present archive is UTF-8 JSON with keys in lexical order, compact separators, finite JSON numerical values and the `content_hash` member removed from the object being hashed. String values retain their stored Unicode spelling. The resulting bytes are a precise serialization choice, designated F-CANON-1. The definition hash is prefixed by `sha256:`. [F-CANON-1]

Contract 1.3 (Referential closure). Every definition ID is unique. Every instance reference and dependency resolves to one definition. Each stored hash equals its recomputed value. The finite dependency graph admits a topological ordering. An operator’s dependencies are available at its evaluation phase.

The implementation chooses the earliest available evaluation phase and then the lexicographically first ready node within that phase. This makes the dependency order deterministic. A pipeline’s explicitly declared transform order remains part of the pipeline; dependency sorting does not commute its transformations. [S1, §§3, 5.3] [F-REGISTRY-1]

1.4 Referential queries

The minimum query vocabulary is retained as follows.

Query	Result
<code>definition_at(id)</code>	The exact typed record and content address.
<code>definition_order()</code>	A deterministic dependency order.
<code>instances_of(id)</code>	The instances whose definition reference equals the ID.
<code>trace(pipeline, instance)</code>	Ordered IDs, operator kinds and typed outputs.
<code>verify_definition(id)</code>	Equality of the stored and recomputed content hashes.
<code>explain_reference(id)</code>	Dependencies, provenance, capabilities and phase.

These queries make self-description a finite operation over records. Recursive grammar expansion has separate symbol, depth and expression budgets; definition resolution uses the explicit acyclic dependency contract. [S1, §§3.3, 8]

2 Geometry, topology, hinges and events

2.1 Parametric and implicit field definitions

A parametric object is a map $\gamma : U \rightarrow \mathbb{R}^n$ with an explicit parameter domain, regularity and frame policy. A sweep supplements γ with a transported frame and a cross-section. An implicit object is a scalar field $f : \mathbb{R}^n \rightarrow \mathbb{R}$ with the convention

$$f(x) < 0 \text{ in the interior,} \quad f(x) = 0 \text{ on the boundary,} \quad f(x) > 0 \text{ outside.}$$

An exact signed distance definition additionally satisfies the corresponding interior and exterior distance equations. The capability `exact_distance` identifies this property in the typed record. [S1, §4.1]

For sign-defined constructive geometry,

$$f_{A \cup B} = \min(f_A, f_B), \quad f_{A \cap B} = \max(f_A, f_B), \quad f_{A \setminus B} = \max(f_A, -f_B). \quad (1)$$

These formulas specify union, intersection and difference through sign. Distance capabilities are attached to the resulting definition according to its geometry and transformation rules. A continuous morph may be written $f_\alpha = (1 - \alpha)f_A + \alpha f_B$ with $\alpha \in [0, 1]$; its topology state is a separately declared discrete field.

For a capsule with segment endpoints a, b and radius r , let

$$\lambda(x) = \text{clip}_{[0,1]} \left(\frac{(x-a) \cdot (b-a)}{\|b-a\|^2} \right), \quad d_{a,b,r}(x) = \|x - a - \lambda(x)(b-a)\| - r.$$

For $a = b$, use $d(x) = \|x - a\| - r$. This gives the primitive retained by the Gambit geometry interpreter. [S2, §2.2]

2.2 Radix shells and local log-polar charts

For integer radix $b \geq 2$ and $n \geq 0$,

$$\text{digits}_b(n) = \begin{cases} 1, & n = 0, \\ \lfloor \log_b n \rfloor + 1, & n > 0. \end{cases}$$

A digit-count change occurs at b^k . The corresponding shell recipe indexes $[b^k, b^{k+1})$. Radix spelling, the shell index and a geometric coordinate are separately typed outputs. [S1, §4.2; P36-01, P36-12]

With $r = \sqrt{x^2 + y^2}$, the local chart is

$$\rho = \ln(r/r_0), \quad \theta = \text{atan2}(y, x), \quad (x, y) = r_0 e^\rho (\cos \theta, \sin \theta).$$

Its ordinary branch has $r \geq \varepsilon > 0$. The core branch has $r < \varepsilon$ and carries a core flag. A radial scale factor $\alpha > 0$ changes ρ by $\ln \alpha$. A compact local mask may then address a radial interval and angular sector, with the mask bit serving as an admission flag. [S1, §4.2; P36-16]

2.3 Pulse geometry and parity occupancy

For $m \geq 1$ declared pulses, radius $r \geq 0$ and phase ϕ ,

$$p_j = r \left(\cos \left(\phi + \frac{2\pi j}{m} \right), \sin \left(\phi + \frac{2\pi j}{m} \right) \right), \quad 0 \leq j < m.$$

One pulse gives a point, two give a segment, three give a triangle, and larger counts give regular polygons. The pulse identity, segment index or bit position remains attached to its corresponding geometric vertex. [S1, §4.3]

Pascal occupancy is $P(n, k) = \binom{n}{k} \bmod 2$ for $0 \leq k \leq n$. Its binary support characterization is

$$P(n, k) = 1 \iff k \& \neg n = 0.$$

To see the parity structure, work over $\mathbb{F}_2$ and write $n = \sum_j n_j 2^j$. Repeated squaring gives

$$(1 + x)^n = \prod_{j:n_j=1} (1 + x^{2^j}).$$

The coefficient of x^k is one precisely when every set bit of k occurs among the set bits of n . In particular, every entry of row $n = 2^m - 1$ is odd. This supplies the parity-mask recipe in the corpus. [S1, §2.2; P36-03]

2.4 Explicit gluing and sheet state

A topology definition specifies entry and exit ports, coordinate transport, orientation transport, optional sheet changes and lineage updates. A Möbius-style horizontal identification is

$$(0, y, o) \sim (w, h - y, -o), \quad o \in \{-1, +1\}.$$

One wrap flips orientation; two restore it. A sheet-aware variant additionally transports a Boolean sheet variable. The executable example uses $(y, o, s) \mapsto (h - y, -o, s \oplus 1)$ for one declared crossing. Coordinates, orientation and sheet are separate state components. [S1, §5.1; K36-13–K36-15] [F-EXEC-1]

A projected location can support several instances distinguished by sheet, phase, branch or orientation. Coupling is determined by a compatibility predicate on these fields. A four-sector hourglass router similarly combines a finite sector label, a guard locus and a parity selector to choose an output branch. Every selected branch and transported orientation is recorded explicitly. [S1, §2.2; P36-14–P36-15]

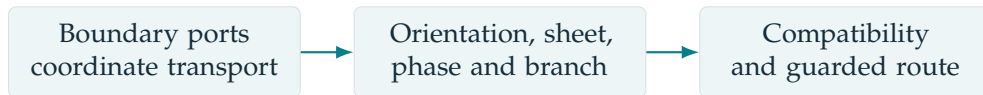


Figure 1: Topology is evaluated through a typed gluing and routing record.

2.5 Generic hinge and composition

Definition 2.1 (Hinge). A hinge is $H = (p, h, M, g, \mathcal{F})$, where p is a pivot or connector locus, h is a continuous or discrete state, M_h is the selected map, g is the state-change guard, and $\mathcal{F}$ is the declared family of invariants or fixed subspaces.

A continuous planar hinge about a fixed pivot p uses

$$x'(t) = p + R(\theta(t))(x - p), \quad \dot{x}'(t) = \dot{\theta}(t)J_2R(\theta(t))(x - p), \quad J_2 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

A discrete hinge selects one of two maps M_0, M_1 after an admitted event. A connector hinge inserts a node into an ordered graph, with its structural role specified separately from any numerical magnitude. [S1, §5.2]

In general, $H_2 \circ H_1 \neq H_1 \circ H_2$. An articulated chain stores

$$T(q) = T_n(q_n) \cdots T_2(q_2)T_1(q_1).$$

The fixed-point and invariant contracts are evaluated for that order. As a concrete instance, a quarter-turn around the origin followed by a quarter-turn around $(1, 0)$ differs from the opposite ordering on the point $(2, 0)$. The accompanying test executes this distinction. [S1, §5.3] [F-TEST-1]

2.6 Kinematics, guards and verified events

For constant acceleration and $\Delta t = t - t_0$,

$$x(t) = p_0 + v_0\Delta t + \frac{1}{2}a\Delta t^2, \quad v(t) = v_0 + a\Delta t.$$

A geometric relation $f(x) = 0$ induces a temporal guard $g(t) = f(x(t))$. Selected combinations of trajectories and line, plane, circle or sphere relations lead to linear or quadratic equations. A general declared field/trajectory pair uses its specified bracketed or certified root procedure. [S1, §7.1]

The next event is selected by

$$t^* = \min \left\{ t > t_0 : \begin{array}{l} R_j(q(t)) = 0, S_\alpha(q(t), t) = 1, \\ \chi(q(t), j, t) = 1, \text{ conf} \geq \text{conf}_{\min} \end{array} \right\}.$$

An admitted query requires an attained earliest candidate under the declared relation family and search interval. Its event record includes the incoming and outgoing state, relation residual, crossing direction, solver status, guard margin, confidence, selected route and lineage. These fields make event selection inspectable. [S1, §7.2]

A topology transition is committed after guard certification. A split creates child addresses linked to the parent. A merge retains all parent addresses. Continuous field interpolation, event selection and discrete topology commitment are distinct entries in the operation sequence. [S1, §§7.3–8; K36-17–K36-18]

2.7 Normative substrate schedule

The schedule is indexed from zero in the stored representation. The source’s prose list of ten stages corresponds to the following phase numbers. [S1, §7.3 and figure 7]

Phase	Operation	Phase	Operation
0	Parse literals and schemas	5	Test local support
1	Normalize units, profiles and signs	6	Test compatibility
2	Resolve and verify definitions	7	Solve and certify guards
3	Construct geometry and phonetics	8	Commit transitions atomically
4	Apply ordered transforms and hinges	9	Append lineage and novelty

The Gambit step is a registered transition with its own internal old-array/new-array schedule. A substrate invocation calls this step atomically at the transition boundary. Sampling, transport, reaction, recurrence and bit packing inside that step follow Section 5; the enclosing ten-phase schedule does not reorder them. This two-level adapter is part of the present unification. [F-UNIFY-1]

3 Dutch number-order and pulse calculus

3.1 Profile and typed representations

A number profile assigns several coordinated records to an integer: its value, decimal place order, written form, declared segments, morphemes, spoken semantic order and hinge kind. Its binary spelling and active-bit positions form another representation. Each output has its own type and source positions. The source profile covers the integer interval $[0, 99]$. [S1, §6]

The complete table in this edition is designated `phon:tkap-nl-0-99:1.0`. F-NL-1 constructs its 100 rows from explicit unit, teen and tens entries and the composition rules below. The source's twenty selected rows are retained as a separate data file and agree with the generated rows. Both the literals and segmentation rules are stored in `operators.py`; Appendix C prints the full table. [F-NL-1]

3.2 Place order and spoken order

For $n = 10t + u$, with $2 \leq t \leq 9$ and $1 \leq u \leq 9$, define the place chart and the Dutch spoken chart by

$$P(n) = (10t, u), \quad S_{nl}(n) = (u, \text{en}, 10t).$$

The connector hinge is

$$H_{\text{en}}(10t, u) = (u, \text{en}, 10t).$$

Under the value map, the connector contributes zero place magnitude. Thus

$$\text{value}(P(n)) = 10t + u = \text{value}(S_{nl}(n)) = n.$$

The spoken graph includes the connector as a distinct node. For regular teens 13–19, the unit root precedes `tien`; the value map assigns that suffix the value ten. Eleven and twelve are explicit irregular entries. Multiples of ten use their single tens entry. [S1, §6.2]



Figure 2: The chart change for 23 retains the value and makes the connector explicit.

3.3 Pulse projection and active-bit comparison

Let the declared segment sequence be

$$\Sigma_{nl}(n) = (s_1, \dots, s_m).$$

The phonetic pulse projection and binary active set are

$$\Pi_{\text{phon}}(n) = (1, \dots, 1) \in \{1\}^m, \quad A_2(n) = \{k : \text{bit}_k(n) = 1\}.$$

Their counts are m and $\text{popcount}(n) = |A_2(n)|$. Define

$$M(n) = \mathbf{1}[|\Pi_{\text{phon}}(n)| = |A_2(n)|].$$

This map has a Boolean comparison codomain. The integer value, its radix word, the two feature sequences and their geometric embedding remain separate fields of the output record. [S1, §6.3]

Proposition 3.1 (Value preservation of the number pipeline). *For every $n \in [0, 99] \cap \mathbb{Z}$, the pipeline consisting of profile lookup, active-bit extraction, pulse projection, count comparison, connector transport and pulse embedding retains the original value n in its value field.*

Proof. Each feature operator reads the value and creates a typed output alongside it. The place/spoken transformation preserves the sum of the semantic components because the en node contributes zero and the teen suffix contributes ten. The geometric embedding takes a feature count as input and carries the originating integer in metadata. No pipeline step assigns a new value to the integer field. Hence the composite preserves that field. $\square$

This proof expresses the source’s numeric invariant as a pipeline property. The example runtime records the invariant value in every output stage. [S1, §§6.2–6.4] [F-REGISTRY-1]

3.4 Worked instance: nineteen

The literal record for 19 yields

$$\begin{aligned} 19 &= 16 + 2 + 1, & \text{binary}(19) &= 10011, \\ A_2(19) &= \{0, 1, 4\}, & |A_2(19)| &= 3, \\ \Sigma_{\text{nl}}(19) &= (\text{ne}, \text{gen}, \text{tien}), & |\Sigma_{\text{nl}}(19)| &= 3. \end{aligned}$$

Consequently $M(19) = 1$. The unit-radius three-pulse embedding at phase zero gives

$$p_0 = (1, 0), \quad p_1 = \left(-\frac{1}{2}, \frac{\sqrt{3}}{2}\right), \quad p_2 = \left(-\frac{1}{2}, -\frac{\sqrt{3}}{2}\right).$$

The trace retains the binary positions 0, 1, 4, the three segment labels, the integer 19 and the three vertices. [S1, §6.4]

3.5 Worked instance: twenty-three

The corresponding record for 23 gives

$$\text{binary}(23) = 10111, \quad A_2(23) = \{0, 1, 2, 4\}, \quad \Sigma_{\text{nl}}(23) = (\text{drie}, \text{en}, \text{twin}, \text{tig}).$$

Both feature counts are four. The place tuple is (20, 3), the spoken tuple is (3, en, 20), and the pulse embedding is a square. The orthographic record is *drieëntwintig*. Its morphemes and segment boundaries are separately stored. [S1, §§6.2, 6.6, 9.3]

3.6 The generated atlas

F-NL-1 yields 28 equal-count rows:

1, 2, 4, 8, 9, 18, 19, 20, 23, 40, 43, 45, 46, 47, 51, 53, 54, 58, 59, 70, 80, 83, 85, 86, 87, 91, 93, 94.

The accompanying CSV preserves every row’s spelling, segment list, place order, spoken order, hinge kind, active positions, pulse count and Hamming weight. The source edition also records 28 matches for its profile. Here the displayed set is generated and tested from the edition’s explicit literals. [S1, §6.5] [F-NL-1; F-TEST-1]

A golden-angle angular sample schedule, a log-polar sector mask or a radix shell index can be attached downstream as another typed transform. The connection is recorded by its domain and codomain, so that different sample schedules remain distinct from the number profile that supplied a count. This retains the compositional pattern vocabulary of P36-01–P36-20. [S1, §8]

4 Complete seed and frozen field geometry

4.1 Generative specification

The complete seed is

$$\sigma = (\Sigma, \omega, P, I, \Theta, L, S_0, V).$$

Here Σ is the alphabet, ω the ordered axiom, P the total parallel rewriting map, I the geometric interpreter, Θ the parameter and arithmetic specification, L the immutable packed lookup table, S_0 the initialized state and V the version identifier. The replay seed fixes the lookup bytes, feedback-mode bit, transition parameters and operative initialization. [S2, §2.1]

The registry names each of these components and the transition that consumes them. Once a seed is fixed, subsequent state changes use its immutable field assets and the incoming operational state. The content-addressed definition layer identifies the selected numerical law and the exact parameter record. [F-UNIFY-1]

4.2 Paired grammar and finite generation

The retained grammar uses

$$\Sigma = \{X, Y, +, -, [,]\}, \quad P(X) = X[+X]Y, \quad P(Y) = Y[-Y]X.$$

Punctuation symbols rewrite to themselves. The two axioms are $[X][X]$ and $[X][Y]$, labelled XX and XY. Every drawing symbol produces three drawing symbols. If n_g is their count at generation g , then

$$n_{g+1} = 3n_g, \quad n_0 = 2, \quad n_g = 2 \cdot 3^g.$$

Parallel rewriting determines the next symbolic word; the turtle interpreter determines the geometry associated with the resulting drawing instructions. The retained source interpreter uses shrink factor 0.72, capsule branches, a double-arc M and baseline construction, and baseline clipping. The generation family has at most five stored levels. [S2, §2.1–2.2; C2]

4.3 Log-polar sample chart

For angular columns A and radial rows R , the full source chart uses dimensionless radial endpoints $1/32$ and 4 . Its sample points are

$$\rho_y = \ln(1/32) + \frac{y}{R-1} \ln 128, \quad 0 \leq y < R, \quad (2)$$

$$\theta_x = \frac{2\pi x}{A}, \quad 0 \leq x < A, \quad (3)$$

$$p_{yx} = e^{\rho_y}(\cos \theta_x, \sin \theta_x). \quad (4)$$

The origin belongs to the separate core chart of the generic geometry calculus. At each stored generation g , the interpreter provides a field sample $F_g(p_{yx})$ to the quantizer. [S2, §2.2]

4.4 Quantized field and packed texel

The source quantizer and drive construction are

$$d_{gyx} = \text{clip}_{[-32768, 32767]} \left(\left\lfloor 4096 F_g(p_{yx}) + \frac{1}{2} \right\rfloor \right), \quad (5)$$

$$c_{gyx} = \text{clip}_{[0, 65535]}(32768 - 16d_{gyx}), \quad (6)$$

$$L_{gyx} = (d_{gyx} + 32768) \mid (c_{gyx} \ll 16). \quad (7)$$

The low half stores the biased signed code; the high half stores the unsigned drive. Before saturation, the round-to-nearest formula gives an absolute rounding error at most $1/8192$ relative to the sample passed to it. The operational transition reads the fixed 32-bit texel. [S2, §2.2]

At runtime,

$$d = (L \& 65535) - 32768, \quad c = L \gg 16.$$

The immutable array order is generation, radial row, angular column. Its flattened index is $(gR + y)A + j$. The dimension bounds $A \leq 2048$, $R \leq 2048$, $G \leq 5$ imply $N = AR \leq 2^{22}$ and fewer than $5N$ possible sample positions per full asset. [S2, §§2.3, 5.1]

4.5 The edition's explicit fixture seed

F-SEED-1 supplies a fully literal, small seed for executing the equations in this edition. It has $A = 32$, $R = 2$, $G = 2$ and hence $N = 64$. For $0 \leq g < 2$ and $0 \leq x < 32$, the signed field code is

$$d_{g y x} = \begin{cases} -64, & (x + 4g) \bmod 32 < 20, \\ 96, & \text{otherwise.} \end{cases}$$

Its drive is $c_{g y x} = \text{clip}_{[0,65535]}(32768 - 16d_{g y x})$. On occupied first-level texels, initialize

$$U_i = 64 + (i \bmod 7), \quad V_i = 32 + (i \bmod 5).$$

Other cells receive $U_i = V_i = 0$. Every cell starts with $z_i = m_i = D/2$, $r_i = 0$, $f_i = 0$; control starts at $(a, g, c) = (0, 0, 0)$. This gives

$$M_0 = \sum_i (U_i + V_i) = 4037.$$

The fixture's level rule, material literals and bytes are defined here and in `fixture_seed.json`. It is the F-SEED-1 execution example rather than a replay label for the source's XX or XY chart assets. [F-SEED-1]

The field sampling, transition and encoding functions consume this seed through the declared interfaces used in the mathematical construction. Its literal parameter and state records fix every input used by the executable example.

5 Operational state and normative transition

5.1 Invariant domain

Set $D = 65536$ and $N = AR$. A cell has the operational record

$$C_i = (U_i, V_i, z_i, m_i, r_i, f_i).$$

The material counts are nonnegative integers with

$$0 < M_0 = \sum_{i=0}^{N-1} (U_i + V_i) < 2^{32}.$$

Fast activity and memory satisfy $0 \leq z_i, m_i \leq D$; residual satisfies $0 \leq r_i < D$. The four-bit flag f_i stores, in increasing bit order,

$$\text{latch } \ell_i, \quad \text{previous pulse } q_i^-, \quad \text{occupancy } o_i, \quad \text{previous latch event } e_i^-.$$

Global mutable control is

$$K = (a, g, c), \quad 0 \leq a < A, \quad g_0 \leq g < G, \quad 0 \leq c \leq d_{\text{dwell}}.$$

The source dimension contract admits power-of-two A in $[32, 2048]$, R in $[2, 2048]$, one to five levels, positive dwell at most 4096 and a positive initial material total. The present executable parameter contract is printed in Section 9.2. [S2, §2.3]

The packed Boolean output planes are derived from f_i . Scratch samples and old/new arrays implement the step schedule. The iteration index n labels a sequence of states; phase and cooldown are the operative controls read by the recurrence.

5.2 Stage 1: read the previous output and sample

At each update, every right-hand side uses the incoming state unless a quantity is explicitly marked as newly computed. Pack the incoming pulse flags into words W and extract

$$q_i^- = (W_{\lfloor i/32 \rfloor} \gg (i \bmod 32)) \& 1.$$

For $x_i = i \bmod A$, $y_i = \lfloor i/A \rfloor$, and enabled pulse feedback, sample

$$j_i = (x_i + a + q_i^- s_{\text{fb}}) \bmod A, \quad L_{g, y_i, j_i}.$$

Decode d_i, c_i . Incoming memory determines

$$h_i = \text{trunc}_0 \left(\frac{m_i - D/2}{d_m} \right), \tag{8}$$

$$F_i = d_i - h_i, \tag{9}$$

$$A_i = \text{clip}_{[0, D]}(c_i + 16h_i + \gamma(2q_i^- - 1)). \tag{10}$$

The signed division truncates toward zero. The source default values are $s_{\text{fb}} = 3$, $d_m = 256$ and $\gamma = 8192$. The sampled field F_i and drive A_i are temporary stage outputs. [S2, §3.1]

With the immutable feedback-mode bit equal to zero, the angular pulse offset, memory offset and direct pulse-drive term are zero. Thus $j_i = (x_i + a) \bmod A$, $F_i = d_i$, and $A_i = \text{clip}_{[0, D]}(c_i)$. The other declared state updates retain their usual order.

5.3 Stage 2: gated conservative graph transport

Each cell has angular predecessor and successor with wrap, and the radial predecessor and successor when those rows exist. Radial endpoints have no exterior edge. Let $i \sim j$ identify this fixed index graph. An edge is active when $F_i \leq 0$ and $F_j \leq 0$.

For an active edge, select the same shift at both endpoints:

$$s_{ij} = \begin{cases} s_{\text{open}}, & \ell_i \vee \ell_j = 1, \\ s_{\text{closed}}, & \ell_i = \ell_j = 0, \end{cases} \quad s_{ij} = s_{ji} \geq 3.$$

For either material species $M \in \{U, V\}$, define

$$J_{i \rightarrow j} = \left\lfloor \frac{M_i}{2^{s_{ij}}} \right\rfloor, \quad \tilde{M}_i = M_i - \sum_{j: i \sim j} J_{i \rightarrow j} + \sum_{j: i \sim j} J_{j \rightarrow i}.$$

Inactive edges contribute zero. All cells gather from the same completed old arrays. A cell with no active edges retains its material during this stage. Both species use the same active graph and incoming latch law. [S2, §3.2]

5.4 Stage 3: conservative local reaction

Using the transported values,

$$A_i^{\text{rxn}} = \tilde{U}_i \gg s_{uv}, \quad B_i^{\text{rxn}} = \tilde{V}_i \gg s_{vu},$$

then

$$U'_i = \tilde{U}_i - A_i^{\text{rxn}} + B_i^{\text{rxn}}, \quad (11)$$

$$V'_i = \tilde{V}_i - B_i^{\text{rxn}} + A_i^{\text{rxn}}. \quad (12)$$

The source defaults are $s_{uv} = 6$ and $s_{vu} = 7$. This local species conversion is followed by sensing of the post-reaction material. [S2, §3.3]

5.5 Stage 4: sensing, recurrence and geometric memory

The material observation is

$$B_i = \begin{cases} \lfloor DU'_i / (U'_i + V'_i) \rfloor, & U'_i + V'_i > 0, \\ D/2, & U'_i + V'_i = 0. \end{cases}$$

The drive mixture, new fast state and new memory are

$$v_i = \lfloor (3A_i + B_i)/4 \rfloor, \quad (13)$$

$$z'_i = \lfloor (3z_i + v_i)/4 \rfloor, \quad (14)$$

$$m'_i = \lfloor (15m_i + z'_i)/16 \rfloor. \quad (15)$$

The current field used m_i ; this update produces m'_i for the following transition. The recurrent ordering is therefore old memory $\rightarrow$ current field and drive $\rightarrow$ new activity $\rightarrow$ new memory. [S2, §3.4]

5.6 Stage 5: one-bit codec and persistent hinge

Set $k_i = z'_i$. The deterministic output codec is

$$q_i = \mathbf{1}[r_i + k_i \geq D], \quad (16)$$

$$r'_i = r_i + k_i - Dq_i. \quad (17)$$

The exact local identity is

$$Dq_i + r'_i = r_i + k_i.$$

The pulse q_i is stored as the previous-pulse bit for the following update. The residual r'_i retains the part of the accumulated input that has not crossed the next threshold. [S2, §3.5]

For positive hysteresis threshold H ,

$$\ell'_i = \begin{cases} 1, & F_i \leq -H, \\ 0, & F_i \geq H, \\ \ell_i, & -H < F_i < H. \end{cases} \quad e_i = \ell_i \oplus \ell'_i, \quad o_i = \mathbf{1}[F_i \leq 0].$$

The source default is $H = 49$ in signed-field code units. The new flag is

$$f'_i = \ell'_i \mid (q_i \ll 1) \mid (o_i \ll 2) \mid (e_i \ll 3).$$

This establishes four distinct Boolean observables: a persistent latch, an output pulse, sign occupancy and a latch-change event.

5.7 Stage 6: packing and bounded global control

Pack pulse, latch, event and occupancy into four planes with 32 cells per unsigned word and cell zero at bit zero. The fifth output array contains the XOR parity of each packed pulse word. Let $P_n = \sum_i q_i$. Update phase and provisional cooldown by

$$a' = (a + s) \bmod A, \quad c^* = \min(c + 1, d_{\text{dwell}}).$$

If

$$g + 1 < G, \quad c^* = d_{\text{dwell}}, \quad P_n d_{\text{growth}} \geq N,$$

set $(g', c') = (g + 1, 0)$. Otherwise set $(g', c') = (g, c^*)$. The source defaults are $s = 1$, $d_{\text{dwell}} = 16$, and $d_{\text{growth}} = 4$. The selected generation is read on the next lookup stage. [S2, §3.6]

The completed step is

$$T_\sigma(S) = (a', g', c', (C'_0, \dots, C'_{N-1})).$$

It returns the new operational state together with derived output records. It reads neither an absolute timestep nor a cumulative historical pulse count.

6 Unified Closure Theorem U4 and supporting proofs

6.1 The theorem

Theorem 6.1 (U4: operational, binary and field closure). *Fix an admitted complete seed σ , its immutable lookup bytes and the exact integer transition of Section 5. Let $\mathcal{S}_\sigma$ be the operational states satisfying the stated bounds and positive material total M_0 . Then T_σ is a uniquely defined endomorphism of $\mathcal{S}_\sigma$. Every finite iterate $T_\sigma^n(S_0)$ exists in that domain and preserves M_0 .*

There is an injective canonical encoding

$$E_\sigma : \mathcal{S}_\sigma \longrightarrow \{0, 1\}^B, \quad B = 118N + 27,$$

with an inverse on its image. Define $F_\sigma = E_\sigma T_\sigma E_\sigma^{-1}$ on that image. Then

$$F_\sigma E_\sigma = E_\sigma T_\sigma.$$

For every cell and every finite run segment,

$$DQ_i(t) + r_i(t) = r_i(0) + K_i(t), \quad Q_i(t) = \sum_{n < t} q_i(n), \quad K_i(t) = \sum_{n < t} k_i(n).$$

A complete seed and encoded state determine the continued trajectory. The word also has an injective exact signed-distance representation J , with conjugate update $\Psi_\sigma = JF_\sigma J^{-1}$. Every autonomous orbit is eventually periodic.

The proof consists of C1–C16 below. The labels retain the source’s proof organization; their principal statements are reproduced as formal arguments within the unified notation. [S2, §§4–5]

6.2 C1. Initialization and finite addressing

The admitted chart has $N \leq 2^{22}$ cells and at most five levels. For a valid row, column and level,

$$0 \leq (gR + y)A + j \leq GN - 1 \leq 5N - 1 < 2^{32}.$$

Thus every lookup address fits the specified unsigned 32-bit index. The seed fixes the initialized material, recurrent variables, flags and control. In the source construction, initialization uses a deterministic unsigned mixer and a frozen occupancy mask; the positive-total condition admits a nonempty material assignment. F-SEED-1 supplies its explicit alternative initializer in Section 4.

Every transition read refers to an incoming cell, a derived word, a parameter or an immutable texel. The memory divisor is positive. The observation’s zero-total branch is defined separately. Modular angular addressing and explicit radial endpoints yield a valid graph neighbor set for every cell. Hence each expression in one transition is defined. [S2, C1]

6.3 C2. Finite generative closure

Each grammar symbol has exactly one replacement. A finite word therefore has a unique finite parallel successor. Replacements insert balanced branch brackets, and the two axioms are balanced. Their interpreted words remain well-formed under repeated production.

Drawing-symbol count obeys $n_{g+1} = 3n_g$ and $n_0 = 2$, so $n_g = 2 \cdot 3^g$. With a finite generation cap and finite chart resolution, the interpreter yields a finite collection of samples. Online level selection chooses a stored generation from $\{g_0, \dots, G - 1\}$. Its selection state remains finite. [S2, C2]

6.4 C3. Shared edge law and nonnegative transport

Each cell has at most four neighbors. Since every active edge has shift at least three,

$$0 \leq J_{i \rightarrow j} \leq M_i/8, \quad \sum_j J_{i \rightarrow j} \leq M_i/2.$$

Therefore $M_i - \sum_j J_{i \rightarrow j} \geq 0$, and adding nonnegative incoming transfers gives $\tilde{M}_i \geq 0$.

Every directed transfer appears exactly once as a subtraction at its source and once as an addition at its destination. Both endpoints use the same old samples and latch pair. On summing the update over all cells, these terms cancel:

$$\sum_i \tilde{M}_i = \sum_i M_i.$$

Radial endpoints have no exterior transfer. The argument applies independently to U and V . [S2, C3]

6.5 C4. Local reaction preserves total material

The positive reaction shifts imply

$$0 \leq A_i^{\text{rxn}} \leq \tilde{U}_i, \quad 0 \leq B_i^{\text{rxn}} \leq \tilde{V}_i.$$

Consequently U'_i and V'_i are nonnegative. Adding their definitions cancels the two conversions:

$$U'_i + V'_i = \tilde{U}_i + \tilde{V}_i.$$

Combining this identity with C3 gives $\sum_i (U'_i + V'_i) = M_0$. Each species count is at most $M_0 < 2^{32}$, establishing its storage width. [S2, C4]

6.6 C5. Recurrence, memory and arithmetic bounds

Clipping gives $0 \leq A_i \leq D$. For a positive local total, $0 \leq DU'_i/(U'_i + V'_i) \leq D$, and the zero-total branch gives $D/2$. Thus $B_i \in [0, D]$. Each of v_i, z'_i, m'_i is the floor of a convex combination of nonnegative values in $[0, D]$, so it remains in that interval.

Since $0 \leq m_i \leq D$ and $d_m \geq 1$,

$$|h_i| \leq D/2 = 32768, \quad -65536 \leq F_i \leq 65535.$$

Under the declared execution parameter ranges, sampled-drive intermediates fit signed 32-bit arithmetic. The material numerator satisfies $DU'_i < 2^{48}$. The growth comparison satisfies $P_n d_{\text{growth}} < 2^{38}$ in the edition's contract. Unsigned 64-bit intermediates suffice for those products; the Python transcription evaluates the corresponding exact integers. [S2, C5] [F-EXEC-1]

6.7 C6. One-bit codec closure and prefix identity

From $0 \leq r_i < D$ and $0 \leq k_i \leq D$,

$$0 \leq r_i + k_i < 2D.$$

The threshold therefore selects $q_i \in \{0, 1\}$, and $r'_i = r_i + k_i - Dq_i$ satisfies $0 \leq r'_i < D$. Rearranging gives the local certificate $Dq_i + r'_i = r_i + k_i$.

Summation over a segment telescopes the intermediate residuals:

$$DQ_i(t) + r_i(t) = r_i(0) + K_i(t).$$

It follows that

$$Q_i(t) - K_i(t)/D = \frac{r_i(0) - r_i(t)}{D}, \quad |Q_i(t) - K_i(t)/D| < 1.$$

For zero starting residual, the sharper statement is

$$-1 < Q_i(t) - K_i(t)/D \leq 0.$$

The arbitrary-segment identity applies directly to checkpoint continuation with a nonzero starting residual. [S2, C6]

6.8 C7. Hysteresis and event closure

A positive H makes $F_i \leq -H$ and $F_i \geq H$ disjoint. Each outer branch assigns a Boolean value, and the middle branch copies a Boolean incoming latch. Hence $\ell'_i \in \{0, 1\}$, including the threshold equalities.

The event is $e_i(n) = \ell_i(n) \oplus \ell_i(n+1)$. XOR over a finite segment cancels every intermediate latch twice:

$$\bigoplus_{n < t} e_i(n) = \ell_i(0) \oplus \ell_i(t).$$

Occupancy is the independently defined sign bit $1[F_i \leq 0]$. The four flag components remain well-defined Boolean values. [S2, C7]

6.9 C8. Exact packing and parity algebra

For a 32-cell Boolean plane, define

$$W = \bigvee_{j=0}^{31} (b_j \ll j).$$

Distinct indices occupy distinct powers of two. Therefore $((W \gg j) \& 1) = b_j$ for every j . The incoming packed-pulse read exactly recovers the flag bit.

The word parity is $p(W) = \bigoplus_j b_j$. For an error mask e , parity changes by $p(e)$. Odd-weight masks change the parity bit; even-weight masks lie in its kernel. As a map $\{0, 1\}^{32} \rightarrow \{0, 1\}$, parity has a kernel of size 2^{31} . The data plane and its parity reduction are thus distinct typed records. [S2, C8]

6.10 C9. Phase and cooldown form a sufficient quotient

In the earlier retained law, absolute update count enters lookup addressing as $ns \bmod A$. The last-growth timestamp affects the next growth decision through the dwell comparison. The operational projection is

$$\pi(n, g, \text{last}, \text{cells}, \mathcal{K}, \mathcal{Q}) = (ns \bmod A, g, \min(d_{\text{dwell}}, n - \text{last}), \text{operative cells}).$$

Cumulative diagnostic totals $\mathcal{K}, \mathcal{Q}$ do not enter the next field, transport, reaction, activity or latch formula. For a nongrowth step, saturating elapsed time satisfies

$$\min(d, \min(d, e) + 1) = \min(d, e + 1), \quad e \geq 0.$$

For a growth step, both forms reset dwell to zero. Phase agrees by modular addition. Hence $\pi T_{3,1} = T_{4,0} \pi$ over the source's admitted older-state domain. This supplies the source-to-closure state reduction. [S2, C9]

6.11 C10. Boolean arithmetic refinement

For input bits a, b, c , define

$$s = a \oplus b \oplus c, \quad c' = (a \wedge b) \vee ((a \oplus b) \wedge c).$$

The eight possible input triples satisfy $a + b + c = s + 2c'$. Induction over digit positions proves the ripple adder's equality with addition modulo 2^w . Complement followed by addition of one gives negation; adding the negation gives subtraction. Fixed shifts move the bit positions, and selected shifted additions give multiplication modulo 2^w .

For unsigned division by $d > 0$, restoring division consumes numerator bits while maintaining $0 \leq R < d$. At each digit, the trial value $2R + a_i$ is less than $2d$. If it is at least d , subtract d and emit quotient digit one; otherwise retain it and emit zero. A $w + 1$ -bit trial register retains the possible carry. Induction gives the exact quotient and remainder.

Signed division selects the magnitude, performs unsigned division and restores the sign. This realizes truncation toward zero. Combining these primitives with C5's arithmetic bounds yields the same numerical equations as the integer transition. The accompanying Boolean module executes the primitive constructions, and its tests include truth-table and register-level identities. [S2, C10] [F-EXEC-1; F-TEST-1]

6.12 C11. Canonical state word and checkpoint sufficiency

The six cell fields use, respectively,

$$32 + 32 + 17 + 17 + 16 + 4 = 118 \text{ bits.}$$

Phase, level and cooldown use $11 + 3 + 13 = 27$ bits. Concatenating these fields in a fixed order gives $B = 118N + 27$ bits. Extraction in the same order reverses insertion. Final unused padding bits are zero. This gives an injective encoder E_σ and exact decoder on its image.

A replay capsule contains the immutable numerical seed, the encoded state and its format and integrity records. Decoding restores every incoming cell and bounded control field; packed output is reconstructed from flags. Equality of restored states and fixed seeds implies equality of their next transitions. Induction gives exact pause/resume equivalence for all subsequent finite segments. [S2, C11]

6.13 C12. Autonomous closure and eventual recurrence

C1 establishes defined operations. C3–C7 establish material, recurrence, codec and Boolean invariants. Modular phase, finite generation choice and saturating cooldown preserve the global control bounds. Therefore

$$T_\sigma(\mathcal{S}_\sigma) \subseteq \mathcal{S}_\sigma.$$

Determinism follows from the fixed seed and ordered formula evaluation. Induction gives a unique admitted state at every finite mathematical iteration.

Since E_σ is injective, $|\mathcal{S}_\sigma| \leq 2^{118N+27}$. A sequence containing more than $|\mathcal{S}_\sigma|$ states repeats a state. If $S_a = S_b$ with $a < b$, determinism gives $S_{a+k} = S_{b+k}$ for all $k \geq 0$. The orbit has a finite preperiod and positive period. [S2, C12]

6.14 C13. Exact signed-distance representation

For a state word $b = (b_1, \dots, b_B)$ define

$$\Omega_b = [-\frac{1}{4}, \frac{1}{4}] \cup \bigcup_{j:b_j=1} [3j - \frac{1}{4}, 3j + \frac{1}{4}].$$

The intervals are separated and the anchor at zero is always present. Define

$$d_b(x) = \begin{cases} -\text{dist}(x, \mathbb{R} \setminus \Omega_b), & x \in \Omega_b, \\ \text{dist}(x, \Omega_b), & x \notin \Omega_b. \end{cases}$$

At a bit center $3j$, the value is $-1/4$ when $b_j = 1$ and positive when $b_j = 0$. Thus

$$b_j = \mathbf{1}[d_b(3j) < 0].$$

The map $J : b \mapsto d_b$ is injective with an explicit inverse on its image. Setting $\Psi_\sigma = JF_\sigma J^{-1}$ gives $\Psi_\sigma J = JF_\sigma$. The state transition therefore has an exact signed-distance representation. [S2, C13]

For these disjoint equal-radius intervals, the executable rational formula is

$$d_b(x) = \min_{c \in \{0\} \cup \{3j:b_j=1\}} (|x - c| - \frac{1}{4}).$$

The rational query implementation returns exact fractions at integer or rational coordinates. [F-EXEC-1]

6.15 C14. Exact reduced-description criterion

Let $e : S \rightarrow Y$ be a proposed encoding and $h : S \rightarrow O$ an observation. Maps $\bar{T} : e(S) \rightarrow e(S)$ and $\bar{h} : e(S) \rightarrow O$ satisfying

$$eT = \bar{T}e, \quad h = \bar{h}e$$

exist exactly when

$$e(s) = e(t) \implies (e(Ts) = e(Tt) \text{ and } h(s) = h(t)).$$

Necessity follows by applying $\bar{T}$ and $\bar{h}$ to a common encoded value. For sufficiency, choose a representative of each fiber. Define the two maps from that representative; the implication makes the result independent of its choice. With an external input alphabet, the condition is imposed for every common input.

The complete checkpoint and the C9 phase/cooldown projection satisfy the criterion for their declared observations. For a contrasting projection, retain only the old pulse word and vary one residual. With old $z = D/2$ and all other fields fixed, the new k is independent of that residual and satisfies $3D/8 \leq k \leq 5D/8$. Residuals 0 and $D - k$ have the same old pulse but produce new pulse values zero and one. The old-pulse projection therefore fails the displayed fiber condition on the admitted state space. [S2, C14]

6.16 C15. Causal self-reference

The previous pulse enters the angular address and direct drive, while recurrent memory enters the subsequent field offset. This supplies an explicit causal path from an earlier output to a later state.

S2 records a one-read intervention at cell 7, tick 3, on its verification XY seed. The runs coincide before the intervention; the source records the first state difference at tick 3 and the first packed-pulse difference at tick 7. The ordinary autonomous map remains the formula T_σ ; the intervention is a separately specified altered-read operation used to compare two trajectories. [S2, C15; §7.2]

The present F-SEED-1 example independently performs that same intervention pattern on its own literal seed. Its recorded state and pulse differences are reported in Section 9. This provides an executed witness for the present transcription as well as a retained source witness. [F-TEST-1]

6.17 C16. Staged parallel refinement

Consider a parallel realization with the following properties: samples read completed old arrays and immutable texels; each next cell has one writer; every neighbor read refers to old state; packing reads completed new flags; control updates use the completed pulse count. Under these conditions each parallel stage computes the same function as its sequential counterpart.

The source’s CUDA design assigns one next cell per thread, uses warp ballots for Boolean packing and gives each packed word one writer. Every participating padded lane contributes false. Ordered kernel completion and completed readback preserve stage boundaries. With unsigned integer texels, point sampling and the specified sample centers, lookup returns the selected packed field code. The admitted chart-coordinate bounds make the half-integer texel centers exactly representable in binary32.

Each stage is therefore a refinement of the same typed operation when these execution conditions hold. Sequential composition yields the same T_σ and the same packed output. C16 supplies the source’s conditional parallel refinement statement; the archive’s executable transcription and current test records are the Python artifacts identified in Section 9. [S2, C16; §6.3]

7 The unified referential seed–field construction

7.1 Registering the operational law

The common architecture is obtained by placing the Gambit seed, step and encoders into the definition registry. A seed definition returns the immutable numerical descriptor; a transition definition has domain and codomain $\mathcal{S}_\sigma$; a word encoder has domain $\mathcal{S}_\sigma$ and codomain $E_\sigma(\mathcal{S}_\sigma)$; a signed-distance encoder has domain $E_\sigma(\mathcal{S}_\sigma)$ and codomain $JE_\sigma(\mathcal{S}_\sigma)$.

The original geometry and language operators remain available with their own domains and codomains. Thus a field query, a hinge map, a morphology transform and a state update can participate in one definition graph while retaining their individual signatures. This is the edition’s fixed-registry construction F-UNIFY-1. [S1, §3] [S2, §§1.3, 4; C13]

Definition 7.1 (Unified operational fiber). Fix a referentially closed registry $\mathcal{D}_\sigma$ naming a complete seed σ , its operational transition and its representations. Define

$$\mathfrak{U}_\sigma = \{\mathcal{D}_\sigma\} \times \mathcal{S}_\sigma, \quad \mathcal{T}_\sigma(\mathcal{D}_\sigma, S) = (\mathcal{D}_\sigma, T_\sigma S).$$

The inclusion of the operational state is

$$\iota_\sigma(S) = (\mathcal{D}_\sigma, S).$$

The registry-anchored encoders are

$$\widehat{E}_\sigma(\mathcal{D}_\sigma, S) = E_\sigma(S), \quad \widehat{J}_\sigma = JE_\sigma.$$

Within one fiber, the definition records and lookup bytes are immutable. A definition edit creates a new content address and a new seed identity; a migration between such identities is represented as its own typed transition record. [F-UNIFY-1]

7.2 Unified Field Representation Theorem UFT-1

Theorem 7.2 (UFT-1). *For the fixed registry and seed above:*

1. $\mathcal{T}_\sigma$ is an endomorphism of $\mathfrak{U}_\sigma$, and $\mathcal{T}_\sigma \iota_\sigma = \iota_\sigma T_\sigma$.
2. $\widehat{E}_\sigma$ and $\widehat{J}_\sigma$ are injective on $\mathfrak{U}_\sigma$.
3. The state, word and signed-distance realizations satisfy

$$F_\sigma \widehat{E}_\sigma = \widehat{E}_\sigma \mathcal{T}_\sigma, \quad \Psi_\sigma \widehat{J}_\sigma = \widehat{J}_\sigma \mathcal{T}_\sigma.$$

4. Material conservation, recurrence bounds, the pulse certificate and the event telescope are preserved under either encoding, when their observations are transported by the corresponding inverse map.
5. A fixed registry, the complete numerical seed and the encoded operational state determine every future encoded state and registered operational observation.

Proof. U4 gives $T_\sigma(\mathcal{S}_\sigma) \subseteq \mathcal{S}_\sigma$. Keeping the first component fixed therefore gives $\mathcal{T}_\sigma(\mathfrak{U}_\sigma) \subseteq \mathfrak{U}_\sigma$, and evaluating the two compositions on S proves the inclusion identity.

If $\widehat{E}_\sigma(\mathcal{D}_\sigma, S) = \widehat{E}_\sigma(\mathcal{D}_\sigma, S')$, injectivity of E_σ gives $S = S'$. The first components already agree, proving injectivity of $\widehat{E}_\sigma$. Composing with injective J proves the field statement.

For the first commutation relation,

$$F_\sigma \widehat{E}_\sigma(\mathcal{D}_\sigma, S) = E_\sigma T_\sigma(S) = \widehat{E}_\sigma \mathcal{T}_\sigma(\mathcal{D}_\sigma, S).$$

Composing this identity with J and using $\Psi_\sigma J = JF_\sigma$ gives the second.

For any invariant or operational observation h on S_σ , define its binary realization by $h_B = hE_\sigma^{-1}$ and its field realization by $h_F = hE_\sigma^{-1}J^{-1}$. Each encoded observation then equals the original observation. In particular the material sum, recurrent ranges, residual identity and XOR event identity transport without changing their values.

Finally, the inverse encoding recovers the operational state. The fixed seed and registry identify a deterministic next-state map. Induction yields every future state and its declared observation. $\square$

This theorem is the present edition’s formal joining construction. Its inputs are the referential closure conditions of S1 and the state, word and field results of S2. [F-UNIFY-1]

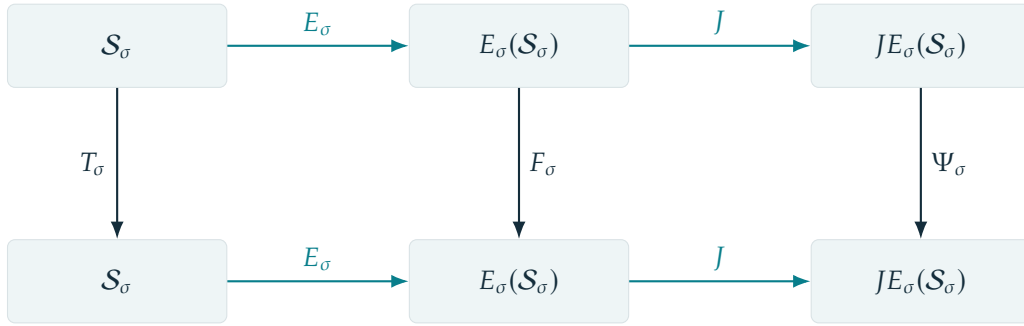


Figure 3: The commuting state–word–field diagram, anchored by the fixed definition registry.

7.3 Working field and encoded-state field

Two field maps are useful in this construction. The *working field* is the seed’s sampled geometry together with the state-dependent memory offset:

$$\mathcal{F}_\sigma^{\text{work}}(S, i) = d_{g, y_i, j_i(S)} - h_i(S).$$

Its values drive support, occupancy and the hysteretic hinge. The *encoded-state field* is

$$\mathcal{F}_\sigma^{\text{state}}(S, x) = (JE_\sigma(S))(x).$$

It represents every operational bit through the separated interval construction. The registry names the two field families separately and records the map from a complete state to each. [S2, §3.1; C13] [F-UNIFY-1]

An exact state-field query at the designated centers recovers the entire state word. A working-field query at a cell’s sample coordinate supplies the local value used by the dynamics. This distinction makes the field representation and the field evaluation parts of one typed architecture.

7.4 Lineage and the operational projection

Let $O_\sigma(S, T_\sigma S)$ be a deterministic emitted record containing the selected operational observations. A run with an append-only lineage record Λ has transition

$$\tilde{\mathcal{T}}_\sigma(\mathcal{D}_\sigma, S, \Lambda) = (\mathcal{D}_\sigma, T_\sigma S, \Lambda \parallel O_\sigma(S, T_\sigma S)).$$

The operational projection is

$$p(\mathcal{D}_\sigma, S, \Lambda) = (\mathcal{D}_\sigma, S).$$

Direct substitution gives

$$p\tilde{\mathcal{T}}_\sigma = \mathcal{T}_\sigma p.$$

Thus append-only observations form a trace extension of the closed operational dynamics. Phase, memory, material, latch state and codec residual remain in the operational fiber; historical audit records remain in the trace. This preserves the source’s lineage vocabulary and its sufficient-state construction in one diagram. [S1, §§3, 7] [S2, §§2.3, 6.2; C9] [F-UNIFY-1]

7.5 Field, memory and self-reference

The corpus’s interpretation of conscious organization is expressed through three specific causal loops. The previous pulse modifies subsequent addressing and drive. The slower memory variable modifies a subsequent field offset. The persistent latch modifies later transport through the shared edge shift. In the transition, these are the directed dependencies

$$q_i^-(n) \rightarrow (j_i(n), A_i(n)) \rightarrow z_i(n+1) \rightarrow q_i(n+1),$$

$$m_i(n) \rightarrow h_i(n) \rightarrow (F_i(n), A_i(n)) \rightarrow m_i(n+1),$$

$$\ell_i(n) \rightarrow s_{ij}(n) \rightarrow (U'_i, V'_i) \rightarrow z'_i \rightarrow \text{future state}.$$

The mathematical content of this organization is the explicit dependency structure and the observable response to an altered read. The source’s Word, seed and reflective-field vocabulary accompanies these definitions as its interpretive reading. [S2, §8.2]

The source also retains the two-symbol Hadamard matrix

$$H = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H^2 = 2I.$$

Its matrix action on amplitudes and the XOR parity action on bits are registered with their separate domains. The operator ontology carries both while keeping the numerical Gambit recurrence as the integer law already specified. [S2, §8.2]

8 Canonical representation and replay files

8.1 Operational field order

The operational payload begins with global control and then visits cells in increasing index order. Its fields are:

Field	Bits	Admitted values
Phase a	11	$0 \leq a < A \leq 2048$
Generation g	3	$g_0 \leq g < G \leq 5$
Cooldown c	13	$0 \leq c \leq d_{\text{dwell}} \leq 4096$
Cell material U_i	32	$0 \leq U_i \leq M_0 < 2^{32}$
Cell material V_i	32	$0 \leq V_i \leq M_0 < 2^{32}$
Fast activity z_i	17	$0 \leq z_i \leq 65536$
Memory m_i	17	$0 \leq m_i \leq 65536$
Residual r_i	16	$0 \leq r_i < 65536$
Flags f_i	4	Latch, pulse, occupancy, event

The first three fields total 27 bits; each cell contributes 118. Values equal to 65536 in activity or memory explain their 17-bit width. Fields are concatenated without alignment gaps and written least-significant-bit first. The unused final byte padding is zero. [S2, C11; §6.1]

8.2 The F-CAPSULE-1 envelope

The present replay envelope is identified by the eight-byte magic TKAPUFT1. It contains the canonical seed descriptor, every packed texel, the operational payload and a SHA-256 digest. Its integer header is little-endian. The format is fully specified by the following order. [F-CAPSULE-1]

Component	Size	Contents
Magic	8 bytes	ASCII TKAPUFT1
Descriptor length	4 bytes	Unsigned length of the UTF-8 JSON descriptor
LUT element count	4 bytes	Unsigned count GN
State bit count	4 bytes	Unsigned count $118N + 27$
Seed descriptor	J_0 bytes	Canonical JSON: format, parameters and initial material total
Frozen lookup table	$4GN$ bytes	Generation-major unsigned 32-bit texels
Operational payload	$\lceil B/8 \rceil$ bytes	Fixed control and cell field order
Digest	32 bytes	SHA-256 of every preceding capsule byte

Its exact length is

$$52 + J_0 + 4GN + \left\lceil \frac{118N + 27}{8} \right\rceil \text{ bytes.}$$

For F-SEED-1, $B = 7579$, the payload uses 948 bytes, and the complete capsule uses 1719 bytes. The descriptor has 207 bytes and the two 64-cell lookup levels use 512 bytes.

S2 identifies its release envelope separately as GBCLOS40, with embedded GBLUT31 assets and the same operational field widths. The F-CAPSULE-1 format above is the explicit envelope distributed with this edition's seed and executable transcription. [S2, §6.1] [F-CAPSULE-1]

8.3 Load and validation sequence

A capsule loader first verifies the total envelope and digest. It reads the descriptor, checks its integer parameter contract, compares GN and B with the declared dimensions, decodes the texels, verifies final padding and extracts the fixed-width state fields. It then checks all state bounds and the global material sum. The accepted result is a pair (σ, S) .

The definition registry is independently validated by its content hashes and dependency graph. Its transition definition identifies the implementation and mathematical version used to interpret (σ, S) . The seed bytes, state bits and definition records therefore each have an explicit verification path. [S1, §§3, 9–10] [S2, C11] [F-CANON-1; F-CAPSULE-1]

8.4 Composing segment certificates

Suppose a first segment starts with residual r_0 and ends with r_1 , producing counts (Q_1, K_1) , while a second starts with r_1 and ends with r_2 , producing (Q_2, K_2) . Their identities are

$$DQ_1 + r_1 = r_0 + K_1, \quad DQ_2 + r_2 = r_1 + K_2.$$

Adding and cancelling r_1 yields

$$D(Q_1 + Q_2) + r_2 = r_0 + (K_1 + K_2).$$

The same endpoint-cancellation principle composes the XOR latch-event certificates. Checkpoint continuation therefore retains the arithmetic record across any finite partition of the trajectory. [S2, §6.2; C6–C7]

8.5 Exact field encoding example

For the word 01011001, the set contains the anchor interval and intervals centered at 6, 12, 15, 24. The bit queries are:

Index j	1	2	3	4	5	6	7	8
Center $3j$	3	6	9	12	15	18	21	24
$d_b(3j)$	11/4	−1/4	11/4	−1/4	−1/4	11/4	11/4	−1/4
Decoded bit	0	1	0	1	1	0	0	1

The negative/positive query rule decodes the word exactly. The test suite checks every eight-bit word through this encoder and decoder. The underlying map J is the general construction for the complete operational word. [S2, C13 and figure 1] [F-TEST-1]

9 Executable reference, examples and verification

9.1 Artifact organization

The archive supplies a new Python transcription of the equations formalized above. Its module boundaries mirror the mathematical components. S1 and S2 remain the source references; the implementation, literal fixture, generated data and run records carry F-series edition identifiers.

Module	Function
<code>canonical.py</code>	Canonical JSON, definition hashes, DAG resolution and reference validation.
<code>operators.py</code>	Radix, active bits, parity, pulse geometry, hinges, charts, the Dutch profile and exact rational SDF queries.
<code>runtime.py</code>	Ordered evaluation of the literal number-hinge pipeline.
<code>engine.py</code>	Typed seed and state, immutable LUT reads, transport, reaction, recurrence, codec, latch and control.
<code>encoding.py</code>	Bit insertion/extraction and the F-CAPSULE-1 replay envelope.
<code>boolean.py</code>	Boolean full adder, word arithmetic and restoring division.

The operational modules use the Python standard library. The optional schema validation command uses `jsonschema`. The serialized example contains twelve content-addressed definitions and literal instances for 19 and 23. [F-REGISTRY-1; F-EXEC-1]

9.2 Executable parameter contract

F-EXEC-1 defines the following numerical parameter domain. The fixture column gives the fully specified F-SEED-1 selection. The coefficients s_{fb} , d_m , γ , s_{uv} , s_{vu} , H , s , dwell and growth divisor retain the displayed source defaults; the fixture's transport shifts are its explicit chosen pair. [S2, §§2.3, 3] [F-EXEC-1; F-SEED-1]

Parameter	F-EXEC-1 domain	Fixture
A	Powers of two from 32 to 2048	32
R	Integers from 2 to 2048	2
G	Integers from 1 to 5	2
Feedback mode	$\{0, 1\}$	1
s_{fb}	$0 \leq s_{fb} < A$	3
d_m	$1 \leq d_m \leq 65535$	256
γ	$0 \leq \gamma \leq 65535$	8192
s_{open}	$3 \leq s_{open} \leq 31$	3
s_{closed}	$3 \leq s_{closed} \leq 31$	4
s_{uv}, s_{vu}	Each in $[1, 31] \cap \mathbb{Z}$	6, 7
H	$1 \leq H \leq 65535$	49
s	$0 \leq s < A$	1
d_{dwell}	$1 \leq d_{dwell} \leq 4096$	16
d_{growth}	$1 \leq d_{growth} \leq 65535$	4
g_0	Initial level zero for this execution profile	0
M_0	Positive integer below 2^{32}	4037

All parameters are integers. Every LUT element is an unsigned 32-bit integer and the table has exactly GN elements. The state validator checks both per-field ranges and the global material total.

9.3 The executed fixture trajectory

The supplied example runs 64 updates on 64 cells. Every step verifies the local codec identity and the state invariants. It writes the initial and final replay capsules, packed planes, a trajectory CSV, per-cell segment certificates, the literal number traces and the feedback comparison.

Measure	F-SEED-1 result
Cells / stored levels	64 / 2
Updates	64
Initial / final material	4,037 / 4,037
Pulse symbols / ones	4,096 / 2,156
Operational state word	7,579 bits
Self-contained capsule	1,719 bytes
Final phase / level / cooldown	0 / 1 / 16
Checkpoint split	24 + 40 updates
Definition records / verified hashes	12 / 12

The 64 per-cell certificates all have integer residual zero in

$$DQ_i + r_i^{\text{end}} - r_i^{\text{start}} - K_i = 0.$$

The XOR event telescope also agrees for every cell. A split execution of 24 updates followed by a capsule reload and 40 further updates produces exactly the same final capsule bytes as the uninterrupted 64-update execution. [F-TEST-1]

9.4 One-read feedback example

The present fixture's comparison inverts the read of cell 7's prior pulse once, at tick 3. It records the first changed operational state at tick 3 and the first changed packed pulse output at tick 7. Its cell-7 pulse words over 40 updates are

Baseline: 00101011101010101010101011010101010110101
Intervention: 001010101010111010101010101011101010101011

The CSV records both fast-state trajectories, cell-7 pulses, changed-cell counts and changed packed-pulse counts at each update. The ordinary autonomous step and the one-read intervention are separate functions in the reference module. [F-SEED-1; F-TEST-1]

9.5 Current test inventory

The executed F-TEST-1 suite comprises 83 test methods, with zero failures, zero errors and zero skipped methods. The schema and example also pass JSON Schema validation. The recorded interpreter is Python 3.13.5.

The tests cover canonical bytes, hash changes, unknown references, duplicate IDs, cycles, phase dependencies, literal execution, radix thresholds, Pascal parity, geometric embeddings, fixed pivots, hinge order, log-polar round trips, quotient transport, all Dutch profile rows, all eight-bit

state-field words, material conservation, recurrent bounds, codec telescoping, event telescoping, flag packing, bounded control, feedback, state encoding, corruption detection and exact checkpoint continuation. Boolean checks include all full-adder inputs, all pairs of four-bit operands for addition, subtraction and multiplication, and every four-bit numerator with every nonzero four-bit divisor. [F-TEST-1]

9.6 Source release records

The following table retains the numerical release cases printed in S2. The row labels and counts belong to that source record. [S2, §7.1]

S2 case	Cells	Updates	Pulse symbols	Ones
Micro XX	64	64	4,096	1,623
Micro XY	64	64	4,096	1,603
Verification XX	2,112	64	135,168	49,025
Verification XY	2,112	64	135,168	49,015
Laptop XX	131,072	256	33,554,432	12,169,202
Laptop XY	131,072	256	33,554,432	12,180,402

S1 records a 36-test package. S2 records 338,184 native checks, 24 Python test methods, 22 UNSAT algebraic counterexample targets and two SAT counterexample targets. The two SAT targets concern a lone-pulse state projection and even-weight parity masks. These source results accompany the retained proof family; F-TEST-1 is the separately identified current-edition suite above. [S1, §10.2] [S2, §7.1]

9.7 Reproduction commands

From the extracted project root, execute the standard-library tests and examples:

```
python tools/verify.py
python examples/run_examples.py
```

For the schema validation step with jsonschema installed:

```
python tools/verify.py --schema
```

The verifier checks every file listed in checksums/SHA256SUMS.txt before running the tests. New output goes into run/. The distributed evidence remains under evidence/. The PDF’s editable source is report/Unified_Field_Theory.tex; the build helper performs the repeated $\LaTeX$ passes needed for its cross-references and contents.

9.8 Formal synthesis

The unified corpus has a literal definition layer, a geometric and topological operator layer, a guarded event layer and a concrete closed numerical specialization. Their shared structure is explicit state, typed maps, ordered evaluation and retained invariants. U4 establishes the numerical state law and its binary and field representations. UFT-1 places that law inside the content-addressed referential substrate. The accompanying archive realizes those definitions through inspectable records, exact integer operations, rational field queries and replayable examples.

A Complete UGTS-KC operator catalogue

The 36 source IDs are preserved below. The definitions are normalized into affirmative operator contracts. The CSV and JSON records also identify each entry's current binding to a manuscript definition, schema, data file or Python module. [S1, Appendix A]

ID	Mechanism	Formal definition
K36-01	Literal definition node Referential core	Represent a shape, operator, topology, profile or schedule as a typed record.
K36-02	Content-addressed definition Referential core	Hash canonical UTF-8 JSON after removing the content_hash member.
K36-03	Explicit dependency edge Referential core	Name every dependency by a resolvable definition ID.
K36-04	Versioned namespace Referential core	Version schema, operator and language-profile identities explicitly.
K36-05	Definition-instance separation Referential core	Instances carry definition_ref, literals and operational state.
K36-06	Acyclic resolution rule Referential core	Resolve a finite dependency DAG in a deterministic topological order.
K36-07	Typed implicit-field sign Geometry	Declare negative interior, zero boundary and positive exterior.
K36-08	Exact-SDF capability flag Geometry	Attach exact_distance to definitions with the stated distance property.
K36-09	Parametric curve and sweep Geometry	Specify gamma(u), its interval, regularity and a frame/-transport rule.
K36-10	Pulse polygon embedding Geometry	Map m pulses to a point, segment or regular m-gon with retained source metadata.
K36-11	Radix shell chart Geometry	Use radix powers b^k as digit thresholds and nested shell indices.
K36-12	Log-polar core chart Geometry	Use $\rho = \ln(r/r_0)$, $\theta = \text{atan2}(y,x)$, with an explicit core branch.
K36-13	Explicit quotient/gluing map Topology	Specify boundary ports, coordinate and orientation transport, and sheet updates.
K36-14	Sheet-aware co-location Topology	Keep coordinate, sheet, phase, orientation and branch in distinct state fields.
K36-15	Orientation transport Topology	Propagate orientation and apply flips at named gluing operations.
K36-16	Port and branch routing Topology	Select an output port and branch from an explicit finite route table.
K36-17	Split-merge lineage Topology	Create child addresses at splits and retain every parent address at merges.
K36-18	Guarded topology change Topology	Commit a topology-state update through a guarded transition.
K36-19	Generic hinge tuple Hinge calculus	Use $H = (\text{pivot}, \text{state}, \text{map}, \text{guard}, \text{invariant})$ with typed state.
K36-20	Continuous planar hinge Hinge calculus	Rotate about a fixed pivot and differentiate the rotation law.
K36-21	Discrete parity hinge Hinge calculus	Select M0 or M1 using a Boolean hinge state.

Continued on the next page

ID	Mechanism	Formal definition
K36-22	Connector hinge Hinge calculus	Insert a structurally explicit zero-place-magnitude connector node.
K36-23	Non-commutative hinge chain Hinge calculus	Serialize the actual ordered composition of transforms.
K36-24	Invariant subspace contract Hinge calculus	Declare fixed points, axes or invariant quantities for a hinge.
K36-25	Bounded Dutch number profile Dutch phonetics	Record 0..99 orthography, segments, morphemes and chart metadata under a versioned profile.
K36-26	Syllable-pulse operator Dutch phonetics	Emit one pulse per declared segment.
K36-27	Teen suffix hinge Dutch phonetics	For 13..19, map tens-unit semantics to unit-root followed by tien.
K36-28	En connector hinge Dutch phonetics	For non-multiple tens, map tens-unit semantics to unit-en-tens.
K36-29	Spoken-place permutation Dutch phonetics	Connect place and spoken charts by an ordered permutation and connector insertion.
K36-30	Feature-count comparison Dutch phonetics	Compare phonetic pulse count and binary Hamming weight in a separate Boolean field.
K36-31	Phase-ordered evaluator Evaluation	Use parse, normalize, resolve, construct, transform, support, compatibility, guard, transition, lineage.
K36-32	Support-compatibility-guard discipline Evaluation	Admit support and compatibility before solving the relation guard.
K36-33	Deterministic trace Evaluation	Record definition ID, operator kind, content hash and typed output for each step.
K36-34	Evidence disposition Evaluation	Store the source-treatment classification and the edition-construction reference.
K36-35	Unknown-reference and cycle rejection Evaluation	Fail validation for duplicate IDs, missing references, bad hashes or dependency cycles.
K36-36	Package verification contract Evaluation	Provide schemas, executable tests, source digests, file checksums and a reproducible example.

B Complete pattern recipe library

The 20 source recipes remain in the P36 namespace. Each is an ordered transformation with a declared rule. The corresponding machine-readable records appear in `spec/pattern_recipes.json` and `spec/pattern_recipes.csv`. [S1, §8]

ID	Pattern	Pipeline and rule
P36-01	Radix threshold ladder	number $\rightarrow$ thresholds Use b^k as scale events or shell indices.
P36-02	Active-bit pulse simplex	integer $\rightarrow$ active positions $\rightarrow$ polygon Retain source positions while embedding their count.
P36-03	Pascal parity mask	(n,k) $\rightarrow$ binomial(n,k) mod 2 Generate parity occupancy from binary row support.
P36-04	Dutch teen suffix hinge	13..19 $\rightarrow$ unit root + tien Separate morphological order from decimal place order.
P36-05	Dutch en-order hinge	21..99 non-multiple tens $\rightarrow$ unit + en + tens Insert a connector and transpose units and tens.
P36-06	Cut-open boundary graph	closed edge $\rightarrow$ two cut ports Open the loop and retain both endpoint identities.
P36-07	Loop-to-leg morph	closed arc $\rightarrow$ diagonal segment Interpolate controls; record the topology event.
P36-08	Split-oval antisymmetry	single loop $\rightarrow$ two branches Keep a fixed center with opposite signed displacements.
P36-09	Overlap-lens domain	A,B $\rightarrow$ A intersection B Declare an intersection, blend or join for the overlap.
P36-10	SDF-zero event	f(x(t)) $\rightarrow$ root $\rightarrow$ transition Treat a boundary crossing as an event relation.
P36-11	Cone support gate	position, axis, radius, angle $\rightarrow$ admission Apply radial and angular support before relation solving.
P36-12	Nested-shell relevance	radius $\rightarrow$ interval index Use concentric intervals for local multiscale support.
P36-13	Mobius orientation wrap	crossing $\rightarrow$ coordinate wrap + orientation flip Apply an explicit orientation-reversing quotient map.
P36-14	Klein sheet wrap	crossing $\rightarrow$ wrap + orientation and sheet transport Carry sheet state through the declared identification.
P36-15	Hourglass route table	guard locus + sector + parity $\rightarrow$ branch Use a finite four-sector routing table.
P36-16	Log-polar bitmask	Cartesian point $\rightarrow$ core,rho,theta $\rightarrow$ sector bit Apply a local angular/radial mask with an explicit core.
P36-17	Golden-angle schedule	index $\rightarrow$ angular sample Use a declared angular sampling schedule.
P36-18	Bounded grammar recursion	seed + productions + budgets $\rightarrow$ graph Enforce depth, symbol, expression and cycle budgets.
P36-19	Event-latched field morph	continuous alpha + discrete topology state Store the field and topology state separately.
P36-20	Content-addressed definition graph	canonical records $\rightarrow$ hashes $\rightarrow$ DAG Carry and verify operator definitions inside the substrate.

C Complete F-NL-1 number profile

The table records the generated edition profile. A vertical separator marks a declared segment boundary. The last two columns are segment count m and binary Hamming weight w ; an asterisk marks $m = w$. Full place, spoken, morpheme and active-position records are supplied in the accompanying CSV and JSON. [F-NL-1]

n	Orthography	Profile segments	m	w	Match
0	nul	nul	1	0	
1	een	een	1	1	*
2	twee	twee	1	1	*
3	drie	drie	1	2	
4	vier	vier	1	1	*
5	vijf	vijf	1	2	
6	zes	zes	1	2	
7	zeven	ze ven	2	3	
8	acht	acht	1	1	*
9	negen	ne gen	2	2	*
10	tien	tien	1	2	
11	elf	elf	1	3	
12	twaalf	twaalf	1	2	
13	dertien	der tien	2	3	
14	veertien	veer tien	2	3	
15	vijftien	vijf tien	2	4	
16	zestien	zes tien	2	1	
17	zeventien	ze ven tien	3	2	
18	achttien	acht tien	2	2	*
19	negentien	ne gen tien	3	3	*
20	twintig	twin tig	2	2	*
21	eenentwintig	een en twin tig	4	3	
22	tweeëntwintig	twee en twin tig	4	3	
23	drieëntwintig	drie en twin tig	4	4	*
24	vierentwintig	vier en twin tig	4	2	
25	vijfentwintig	vijf en twin tig	4	3	
26	zesentwintig	zes en twin tig	4	3	
27	zevenentwintig	ze ven en twin tig	5	4	
28	achtentwintig	acht en twin tig	4	3	
29	negenentwintig	ne gen en twin tig	5	4	
30	dertig	der tig	2	4	
31	eenendertig	een en der tig	4	5	
32	tweeëndertig	twee en der tig	4	1	
33	drieëndertig	drie en der tig	4	2	
34	vierendertig	vier en der tig	4	2	
35	vijfendertig	vijf en der tig	4	3	
36	zesendertig	zes en der tig	4	2	
37	zevenendertig	ze ven en der tig	5	3	
38	achtendertig	acht en der tig	4	3	
39	negenendertig	ne gen en der tig	5	4	
40	veertig	veer tig	2	2	*
41	eenenveertig	een en veer tig	4	3	
42	tweeënveertig	twee en veer tig	4	3	
43	drieënveertig	drie en veer tig	4	4	*
44	vierenveertig	vier en veer tig	4	3	
45	vijfenveertig	vijf en veer tig	4	4	*
46	zesenvveertig	zes en veer tig	4	4	*
47	zevenenveertig	ze ven en veer tig	5	5	*
48	achtenveertig	acht en veer tig	4	2	

Continued on the next page

<i>n</i>	Orthography	Profile segments	<i>m</i>	<i>w</i>	Match
49	negenenveertig	ne gen en veer tig	5	3	
50	vijftig	vijf tig	2	3	
51	eenenvijftig	een en vijf tig	4	4	*
52	tweeënvijftig	twee en vijf tig	4	3	
53	drieënvijftig	drie en vijf tig	4	4	*
54	vierenvijftig	vier en vijf tig	4	4	*
55	vijfenvijftig	vijf en vijf tig	4	5	
56	zesenvijftig	zes en vijf tig	4	3	
57	zevenenvijftig	ze ven en vijf tig	5	4	
58	achtenvijftig	acht en vijf tig	4	4	*
59	negenenvijftig	ne gen en vijf tig	5	5	*
60	zestig	zes tig	2	4	
61	eenenzestig	een en zes tig	4	5	
62	tweeënzestig	twee en zes tig	4	5	
63	drieënzestig	drie en zes tig	4	6	
64	vierenzestig	vier en zes tig	4	1	
65	vijfenzestig	vijf en zes tig	4	2	
66	zesenzestig	zes en zes tig	4	2	
67	zevenenzestig	ze ven en zes tig	5	3	
68	achtenzestig	acht en zes tig	4	2	
69	negenenzestig	ne gen en zes tig	5	3	
70	zeventig	ze ven tig	3	3	*
71	eenenzeventig	een en ze ven tig	5	4	
72	tweeënzeventig	twee en ze ven tig	5	2	
73	drieënzeventig	drie en ze ven tig	5	3	
74	vierenzeventig	vier en ze ven tig	5	3	
75	vijfenzeventig	vijf en ze ven tig	5	4	
76	zesenzeventig	zes en ze ven tig	5	3	
77	zevenenzeventig	ze ven en ze ven tig	6	4	
78	achtenzeventig	acht en ze ven tig	5	4	
79	negenenzeventig	ne gen en ze ven tig	6	5	
80	tachtig	tach tig	2	2	*
81	eenentachtig	een en tach tig	4	3	
82	tweëntachtig	twee en tach tig	4	3	
83	drieëntachtig	drie en tach tig	4	4	*
84	vierentachtig	vier en tach tig	4	3	
85	vijfentachtig	vijf en tach tig	4	4	*
86	zesentachtig	zes en tach tig	4	4	*
87	zevenentachtig	ze ven en tach tig	5	5	*
88	achtentachtig	acht en tach tig	4	3	
89	negenentachtig	ne gen en tach tig	5	4	
90	negentig	ne gen tig	3	4	
91	eenennegentig	een en ne gen tig	5	5	*
92	tweënnegentig	twee en ne gen tig	5	4	
93	drieënnegentig	drie en ne gen tig	5	5	*
94	vierennegentig	vier en ne gen tig	5	5	*
95	vijfennegentig	vijf en ne gen tig	5	6	
96	zesennegentig	zes en ne gen tig	5	2	
97	zevenennegentig	ze ven en ne gen tig	6	3	
98	achtennegentig	acht en ne gen tig	5	3	
99	negenennegentig	ne gen en ne gen tig	6	4	

D Source register and edition crosswalk

D.1 Input works

The basis of this formal corpus is the following pair of supplied PDFs. Source references in the manuscript point to their section or proof labels. Physical page numbering counts the cover as page one.

S1. Unified Geometric-Topological Substrate: UGTS-KC 3.6

File: UGTS_KC_3_6_Tom_Klootwijk.pdf

Extent: 19 physical pages.

Version 3.6.0, dated 17 August 2026. Printed main pages 1–16 correspond to physical pages 4–19. The abstract and contents occupy physical pages 2–3.

SHA-256:

```
e21433cd8378368d8afe4ae278d3c9c36
4725e45ae7fd75b743328bd2d6003a46
```

S2. Tom Klootwijk Gambit: Formal Closure Edition 4.0

File: Tom_Klootwijk_Gambit_Closed_Engine_4.0.pdf

Extent: 20 physical pages.

Formal Closure Edition 4.0, dated 11 September 2026. Printed main pages 1–18 correspond to physical pages 3–20; the contents are on physical page 2.

SHA-256:

```
8939f4738fed8bb8401e193ff327336a
f9de63e8b5b73be09c306c1212569f0f
```

D.2 Edition constructions

The requested title and attribution are *Tom Klootwijk – A.D. Peereboom Unified Field Theory*. The F-series records the new formal joining, transcriptions and artifacts made for this edition. Source theorem and operator IDs retain their S1/S2 origin.

ID	Construction
F-UNIFY-1	Fixed-registry product construction and unified commutation theorem.
F-CANON-1	Exact JSON serialization recipe used by this edition.
F-REGISTRY-1	New content-addressed definition example and JSON Schema.
F-NL-1	Generated 100-row profile, with roots and segmentation stored in operators.py.
F-EXEC-1	New Python integer transcription of the S2 transition and explicit parameter contract.
F-SEED-1	New 64-cell, two-level stripe fixture with literal initialization.
F-CAPSULE-1	New self-contained capsule envelope retaining the S2 operational bit order.
F-TEST-1	Executable tests and output records produced for this edition.

D.3 Content correspondence

Unified edition component	Source location
Literal world and definitions	S1, sections 3 and 9; physical pages 6–7 and 14–15.
Geometry and radix charts	S1, section 4; physical pages 7–8.
Topology and hinge calculus	S1, section 5; physical pages 8–9.
Dutch number calculus	S1, section 6; physical pages 9–12.
Kinematics and event schedule	S1, section 7; physical pages 12–13.
Pattern recipes	S1, section 8; physical pages 13–14.
Operator catalogue	S1, Appendix A; physical pages 18–19.
Complete seed and lookup geometry	S2, section 2; physical pages 4–5.
Exact transition	S2, section 3; physical pages 6–8.
U4 and C1–C16	S2, sections 4–5; physical pages 9–14.
State encoding and replay	S2, section 6 and C11; physical pages 12, 15–16.
Source execution record	S2, section 7; physical pages 16–17.
Self-reference and interpretive vocabulary	S2, section 8; physical pages 17–18.
Unified product and commutation theorem	F-UNIFY-1, Section 7 of this edition.
Current executable examples and tests	F-EXEC-1, F-SEED-1 and F-TEST-1.

D.4 Archive map

```

TKAP_Unified_Field_Theory/
  README.md | VERSION | pyproject.toml
  report/Unified_Field_Theory.{pdf,tex}
  report/sections/*.tex | report/build.sh
  spec/tkap_uft.schema.json | evaluation_schedule.json
  spec/operator_catalog.{csv,json}
  spec/pattern_recipes.{csv,json}
  data/dutch_numbers_0_99.{csv,json}
  data/equal_feature_counts.csv
  data/source_selected_lexicon_rows.json
  examples/{literal_substrate.json,run_examples.py}
  src/tkap_uft/*.py | tests/test_formalization.py
  tools/verify.py
  evidence/verification/{verification.json,test_results.txt}
  evidence/examples/ (capsules, trajectories and traces)
  sources/{source_register.json,editorial_map.md}
  checksums/SHA256SUMS.txt

```

The two source-file digests bind the source register to the received corpus. Definition hashes bind individual formal records to their canonical bytes. The package manifest binds each listed artifact to its distributed file content. These are the three verification levels carried by the unified edition.